

Lab 1: Using GPIO as Input and Output

Katelyn Nguyen
Oluwasegun Oluwole
CSCE 462 - 500
Group 1

22 August 2024

Hardware Description

Provide a written description of your hardware **including a wiring diagram**. Wiring diagrams can be screenshots from [multisim](#) (online or desktop applications), [circuitio.io](#), or a **LEGIBLE** photo of a hand-drawn diagram.

Description:

This lab's hardware consisted of a 16GB microSD card, 2 Common Cathode RGB LEDs, 1 Raspberry Pi 3B+, 1 push button, a 7-segment display, and multiple resistors to prevent LED burnout.

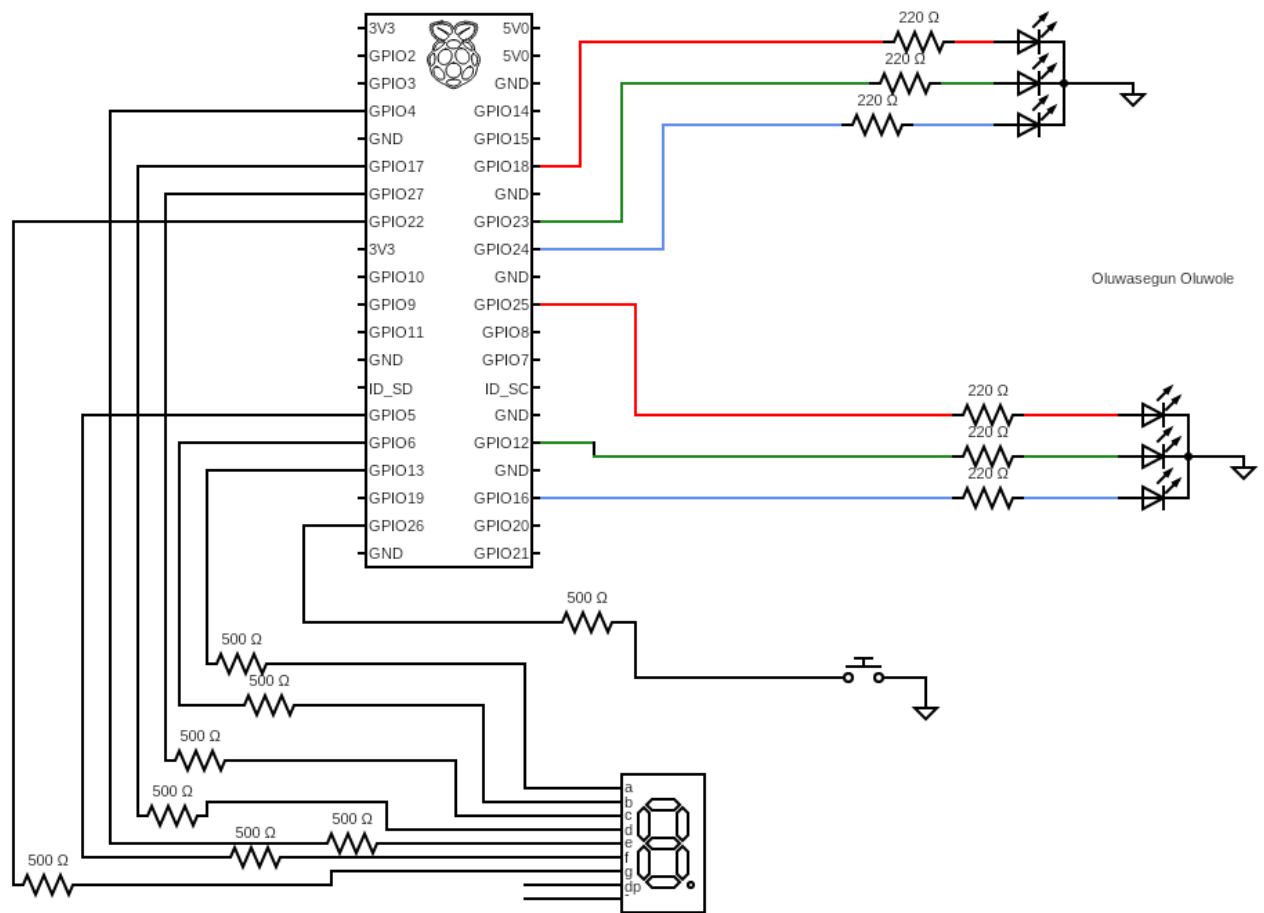
The 16GB microSD card contained the image of the Raspberry Pi OS which was inserted into the raspberry pi and booted up once the raspberry pi's power source was connected. Furthermore, the GPIO pins allowed for the connection of additional devices to the raspberry pi and the use of the raspberry pi as the brain of the circuit.

The 7-segment display serves as the countdown for the system. Pin A of the 7-segment display is connected to a 500 ohm resistor which is then connected to GPIO13. Pin B is connected to a 500 ohm resistor which is then connected to GPIO6. Pin C is connected to a 500 ohm resistor which is then connected to GPIO27. Pin D is connected to a 500 ohm resistor which is then connected to GPIO17. Pin E is connected to a 500 ohm resistor which is then connected to GPIO4. Pin F is connected to a 500 ohm resistor and then to GPIO5.

The RGB LED in the top right corner of the wiring diagram below represents traffic light 1. The pin for red is connected to a 220-ohm resistor which is then connected to GPIO18. The pin for the cathode is connected directly to the ground. The pin for green is connected to a 220 ohm resistor which is then connected to GPIO23. The pin for blue is connected to a 220 ohm resistor which is then connected to GPIO24.

The 2nd RGB LED represents traffic light 2. The pin for red is connected to a 220 ohm resistor which is connected to GPIO25. The pin for the cathode is connected directly to the ground. The pin for green is connected to a 220 ohm resistor which is connected to GPIO12. The pin for blue is connected to a 220 ohm resistor which is connected to GPIO16.

The push button triggers the system to begin its flow of execution. The input rail for the push button is connected to a 500 ohm resistor which is connected to GPIO26. The output rail for the push button is connected directly to the ground.



Software Description

Provide a description of your code and its implementation. **Remember to include your code in your submissions as a separate file. DO NOT PROVIDE A ZIP, TAR, or other file format as your submission unless specified.**

Csce462-500_Group1_Lab1_polling.py

Description 1:

The below code is assigning components of the system to GPIO pins and then sets up the pins as either output or input. In addition, for the RGB LEDs and the 7 segment display, it sets their initial output values as low. The button, when it is not pressed is pulled up to logic high via a pull-up resistor.

```
import time
```

```
import RPi.GPIO as GPIO
```

LABEL ALL THE BCM GPIO PIN NUMBERS AND THE CORRESPONDING COMPONENT

traffic light 1:

red1 = 18

green1 = 23

blue1 = 24

traffic light 2:

red2 = 25

green2 = 12

blue2 = 16

seven segment:

segE = 4

segD = 17

segC = 27

skipping the decimal point

segG = 22

segF = 5

segA = 13

segB = 6

button

button = 26

GPIO SETTINGS

GPIO.setmode(GPIO.BCM)

GPIO.setup(red1, GPIO.OUT, initial=GPIO.LOW)

GPIO.setup(green1, GPIO.OUT, initial=GPIO.LOW)

GPIO.setup(blue1, GPIO.OUT, initial=GPIO.LOW)

GPIO.setup(red2, GPIO.OUT, initial=GPIO.LOW)

GPIO.setup(green2, GPIO.OUT, initial=GPIO.LOW)

GPIO.setup(blue2, GPIO.OUT, initial=GPIO.LOW)

GPIO.setup(segA, GPIO.OUT, initial=GPIO.LOW)

GPIO.setup(segB, GPIO.OUT, initial=GPIO.LOW)

GPIO.setup(segC, GPIO.OUT, initial=GPIO.LOW)

GPIO.setup(segD, GPIO.OUT, initial=GPIO.LOW)

GPIO.setup(segE, GPIO.OUT, initial=GPIO.LOW)

GPIO.setup(segF, GPIO.OUT, initial=GPIO.LOW)

GPIO.setup(segG, GPIO.OUT, initial=GPIO.LOW)

```
GPIO.setup(button, GPIO.IN, pull_up_down=GPIO.PUD_UP)
```

Description 2:

The below code maps the seven segment display values to the segments for each number. Then there are helper functions for the RGB LEDs that pertain to all the states that will be needed for this lab.

SEVEN SEGMENT DISPLAY HELPER DICTIONARIES

```
num = {
    ' ': (0, 0, 0, 0, 0, 0, 0),
    '0': (1, 1, 1, 1, 1, 1, 0),
    '1': (0, 1, 1, 0, 0, 0, 0),
    '2': (1, 1, 0, 1, 1, 0, 1),
    '3': (1, 1, 1, 1, 0, 0, 1),
    '4': (0, 1, 1, 0, 0, 1, 1),
    '5': (1, 0, 1, 1, 0, 1, 1),
    '6': (1, 0, 1, 1, 1, 1, 1),
    '7': (1, 1, 1, 0, 0, 0, 0),
    '8': (1, 1, 1, 1, 1, 1, 1),
    '9': (1, 1, 1, 0, 0, 1, 1)
}

seg_map = {0: segA, 1: segB, 2: segC, 3: segD, 4: segE, 5: segF, 6: segG}
```

TRAFFIC LIGHT 1 LED HELPER FUNCTIONS

```
def traffic1_off():
    GPIO.output(red1, GPIO.LOW)
    GPIO.output(blue1, GPIO.LOW)
    GPIO.output(green1, GPIO.LOW)
```

```
def traffic1_red():
    GPIO.output(red1, GPIO.HIGH)
    GPIO.output(blue1, GPIO.LOW)
    GPIO.output(green1, GPIO.LOW)
```

```
def traffic1_blue():
    GPIO.output(red1, GPIO.LOW)
    GPIO.output(blue1, GPIO.HIGH)
    GPIO.output(green1, GPIO.LOW)
```

```

def traffic1_green():
    GPIO.output(red1, GPIO.LOW)
    GPIO.output(blue1, GPIO.LOW)
    GPIO.output(green1, GPIO.HIGH)

# TRAFFIC LIGHT 2 LED HELPER FUNCTIONS
def traffic2_off():
    GPIO.output(red2, GPIO.LOW)
    GPIO.output(blue2, GPIO.LOW)
    GPIO.output(green2, GPIO.LOW)

def traffic2_red():
    GPIO.output(red2, GPIO.HIGH)
    GPIO.output(blue2, GPIO.LOW)
    GPIO.output(green2, GPIO.LOW)

def traffic2_blue():
    GPIO.output(red2, GPIO.LOW)
    GPIO.output(blue2, GPIO.HIGH)
    GPIO.output(green2, GPIO.LOW)

def traffic2_green():
    GPIO.output(red2, GPIO.LOW)
    GPIO.output(blue2, GPIO.LOW)
    GPIO.output(green2, GPIO.HIGH)

def traffic2_blue_blinking():
    # blink 3 times
    for _ in range(3):
        traffic2_blue()
        time.sleep(.1)
        traffic2_off()
        time.sleep(1)

```

Description 3:

The function below pertains to the countdown from 9 to 0 as well as the blinking that occurs in traffic light 1 from 4 to 0 seconds and then turns off the display after the countdown is completed.

```

def seg_on():
    # The seven segment display counts down from 9 to 0
    for i in range(9, -1, -1):

        # when the countdown is at 4, 3, 2, or 1, traffic light 1 will blink blue
        if (i == 4 or i == 3 or i == 2 or i == 1): # TIME IS 4s
            traffic1_blue()
            time.sleep(.1)
            traffic1_off()
            time.sleep(.1)

        # when the countdown reaches 0,
        # traffic light 1 turns red, and traffic light 2 turns green
        if i == 0: # TIME IS 0s
            traffic1_red()
            traffic2_green()

        # Enables and disables the proper segments depending on the countdown value
        for j in range(7):
            if num[str(i)][j] == 1:
                GPIO.output(seg_map[j], GPIO.HIGH)
            else:
                GPIO.output(seg_map[j], GPIO.LOW)
            time.sleep(1)

        # Turn off the display after countdown is completed
        for j in range(7):
            GPIO.output(seg_map[j], GPIO.LOW)

```

Description 4:

In the main function the polling occurs via the while loop. Upon entering main, light one will be red and light 2 will be green. When the button is pressed, the flow of execution for the traffic light system will take place and finally there is a “GPIO.cleanup()” to clean up all ports used in the program. The time.sleep(5) is to ensure that another valid press does not occur for 20 seconds from the time the button is pressed.

```

def main():
    # TESTING THE COLORS
    # cycle_rgb1()

```

```

# cycle_rgb2()

print("starting")
try:
    while True:
        # Continuously check if the button has been pressed
        time.sleep(0.1)

        # Execute the following actions if the button is/isn't pressed
        if GPIO.input(button) == GPIO.LOW:
            # ~ print("button is pressed")
            traffic2_blue()
            traffic2_blue_blinking()
            traffic2_red()
            traffic1_green()
            seg_on()
            time.sleep(5)
        else:
            # ~ print("button is not pressed")
            traffic2_green()
            traffic1_red() #WE ADDED THIS

finally:
    # Clean the GPIO pin settings for a future run, no matter how the script ended
    GPIO.cleanup()

if __name__ == "__main__":
    main()

GPIO.cleanup()

```

Description 5:

In the implementation of the system where interrupt is used instead of polling, the setup code remains the same as that of seen in the code where polling is used. Below is the `button_callback` which is the action taken when the button is pressed. Since the button is pulled up to high via a pull up resistor the event detection occurs on the falling edge(when the button's state goes from high to low) triggering the `button_callback` to take place which is the flow of execution for the traffic light system. The `bouncetime` in the `add_event_detect` is set to 50 to ensure that another valid press cannot occur for 20 seconds from the time the button is pushed. Finally, `GPIO.cleanup()` is used to clean up all the ports used during the program execution.


```

def button_callback(button):
    # This is the actions that should take place after the button is pressed
    # This will only trigger if the button is pressed

    time.sleep(0.1)
    if GPIO.input(button) == GPIO.LOW:
        # ~ print("button is pressed")
        traffic2_blue()
        traffic2_blue_blinking()
        traffic2_red()
        traffic1_green()
        seg_on()
        time.sleep(5)
    else:
        # ~ print("button is not pressed")
        traffic2_green()
        traffic1_red() #WE ADDED THIS

def main():
    # TESTING THE COLORS
    # cycle_rgb1()
    # cycle_rgb2()

    print("starting")
    try:
        # When the python script is started, these actions occur

        time.sleep(1)
        traffic2_green()
        traffic1_red() #WE ADDED THIS

        # Created an event detection for the falling edge of the button, specifying functionality in
        button_callback()
        GPIO.add_event_detect(button,
                               GPIO.FALLING,
                               callback=button_callback,
                               bouncetime=50)

        # Need a continuous while loop to check for button activity
        while True:
            pass

```

finally:

```
# Clean the GPIO pin settings for a future run, no matter how the script ended
GPIO.cleanup()
```

```
if __name__ == "__main__":
    main()
```

```
GPIO.cleanup()
```

Lab Questions

Provide the **questions and answers** to the questions included in the lab manual.

1. What is RPI 3b+ CPU clock speed? Why is it important to know?
 - a. The clock speed for RPI 3b+ is 1.4 GHz. It is important to know the clock speed so one has an idea of the processing power of the raspberry pi.
2. What is the range of voltage that represents logic low on Raspberry Pi?
 - a. The range of voltage that represents logic low is 1V to 3.3 V.
3. What is the range of voltage that represents logic high on Raspberry Pi?
 - a. The range of voltage that represents logic high is 3.3V to 5V.
4. Compare the Raspberry Pi 3b+ with a standard PC, can you use Pi as a desktop PC? What's an application for RPi?
 - a. The Pi can be used as a desktop PC but is more suited for light tasks such as web browsing and word processing. An application for RPi is as a game server.
5. Discuss the difference(s) between polling and interrupt. What situation should we use polling and what situation should we use interrupt?
 - a. Polling is continuously checking a flag until its condition changes. Interrupt allows devices to promptly signal for attention from the CPU. Polling results in wasted cpu cycles while interrupt reduces wasted cycles by only signaling CPU when necessary. Polling should be used when constant checking of a device is required or in a system that has fewer devices; Interrupt should be used when devices require less frequent attention.
6. What is a hardware interrupt? What is a software interrupt?
 - a. A hardware interrupt is caused by a I/O requests from external devices or hardware failure. Software interrupt occurs when software signals

processor that it needs attention. Software interrupts are how programs request a service from the operating system or to signal an event. In addition hardware interrupts are asynchronous meaning the CPU is interrupted at any time to handle an event, while software interrupts are synchronous meaning the interrupt is generated by executing specific instructions.